

Apresentação do Projeto TimerBook

Equipe: Lucas Monteiro, Matheus Silva, Tiago Barbosa, Vitor Kawan, Kayke Zago, Bruno.

Disciplina: Projeto de Sistemas.

Professor: Edeilson Milhomem da Silva.

1





Ideia do Produto

O projeto nasceu da identificação de um problema real vivenciado pelo público-alvo. A solução proposta busca entregar valor concreto por meio de uma abordagem simples, eficiente e centrada no usuário.



Problema Identificado

Dificuldades para realizar, organizar e monitorar suas leituras.



Público-Alvo

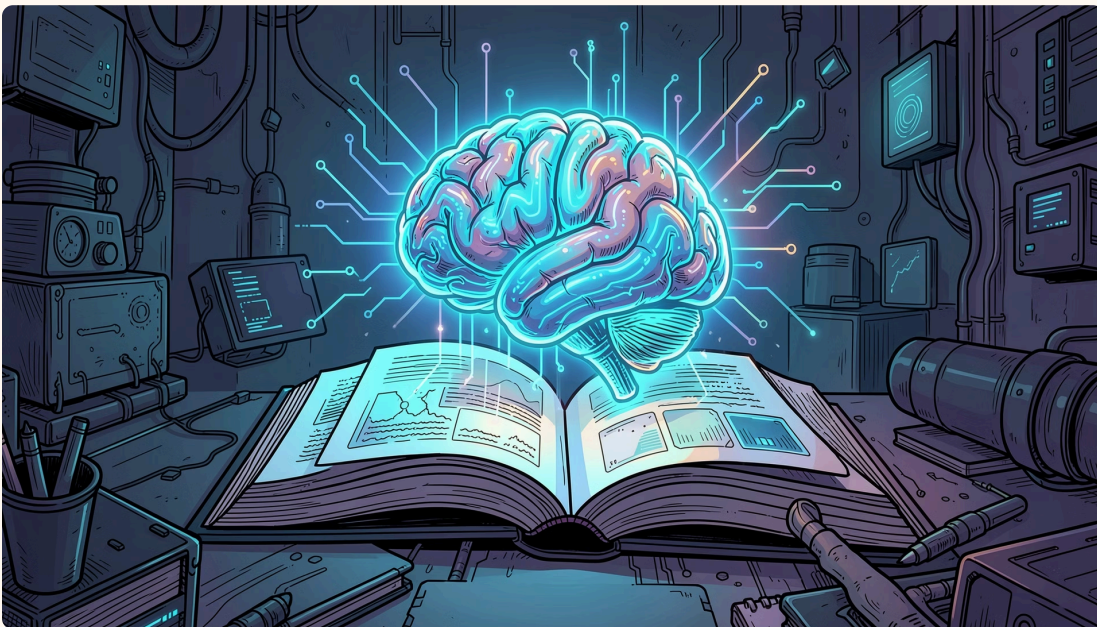
Estudantes e leitores ativos nas faixas de 14 a 50 anos.



Proposta de Solução

TimerBook centraliza o processo de leitura e aprendizado de forma simples e eficiente.

Leitura inteligente, hábito construído



Biblioteca Pessoal Integrada

Adicione e leia seus próprios livros em um leitor web e mobile unificado.

IA com Contexto Ativo

Encontrou um parágrafo difícil? A IA resume e explica aquela página exata, na hora.

Gamificação do Hábito

Metas diárias, conquistas e relatórios de desempenho mantêm o usuário engajado.

MVP Entregue e Escopo

Funcionalidades Implementadas

- Autenticação e controle de acesso de usuários
- Dashboard principal com visão geral dos dados
- Cadastro, edição e exclusão de registros
- Gerenciamento de dos livros pdf(deleção, exclusão e leitura)
- I.A integrada ao leitor
- Interface responsiva para mobile e desktop

O MVP contempla as funcionalidades essenciais para validar a proposta de valor junto ao público-alvo.

Requisitos e Validação

Os requisitos foram levantados por meio de entrevistas e pesquisa com o público-alvo, priorizados com base no valor entregue e na viabilidade técnica dentro do prazo do projeto.

Valor do Produto

O valor principal fica claro logo no primeiro acesso: foco e economia de tempo. Quando o usuário entra no sistema, ele não perde tempo navegando por menus poluídos. Ele encontra um painel que vai direto ao ponto, entregando atalhos para continuar exatamente de onde parou e acesso rápido à biblioteca. O valor está em eliminar qualquer obstáculo entre a vontade de ler e a leitura de fato.

O TimerBook resolve o problema de quem perde o controle das leituras ou precisa de incentivo para manter o hábito. Isso é entregue de três formas:

Leitura rápida

A barra lateral tem uma seção dinâmica de "Recentes". Com um clique, a pessoa retoma a leitura de onde parou, sem perder tempo procurando o livro.

Progresso visível

Botões de "Ver estatísticas" direto nos cartões da biblioteca. Isso ajuda o usuário a ver o próprio avanço em dados reais, o que motiva a continuar lendo.

Organização fácil

O processo de adicionar, editar ou excluir um livro é bem visual e simples de usar, mantendo a biblioteca pessoal sempre organizada.

Inovação e Originalidade

O TimerBook foi desenvolvido com uma abordagem inovadora para unir organização, rotina e experiência do usuário em uma solução simples, intuitiva e diferenciada.

Análise de concorrentes

Comparação realizada com parâmetros claros, como usabilidade, organização, automação e experiência geral do usuário.

Exemplo concreto

Centralização de informações essenciais para reduzir etapas e tornar a experiência mais rápida.

Elementos inovadores

Interface objetiva, navegação fluida e recursos pensados para facilitar o uso no dia a dia, com foco em praticidade.

Exemplo concreto

Organização visual que ajuda o usuário a identificar prioridades com clareza imediata.

Diferenciais competitivos

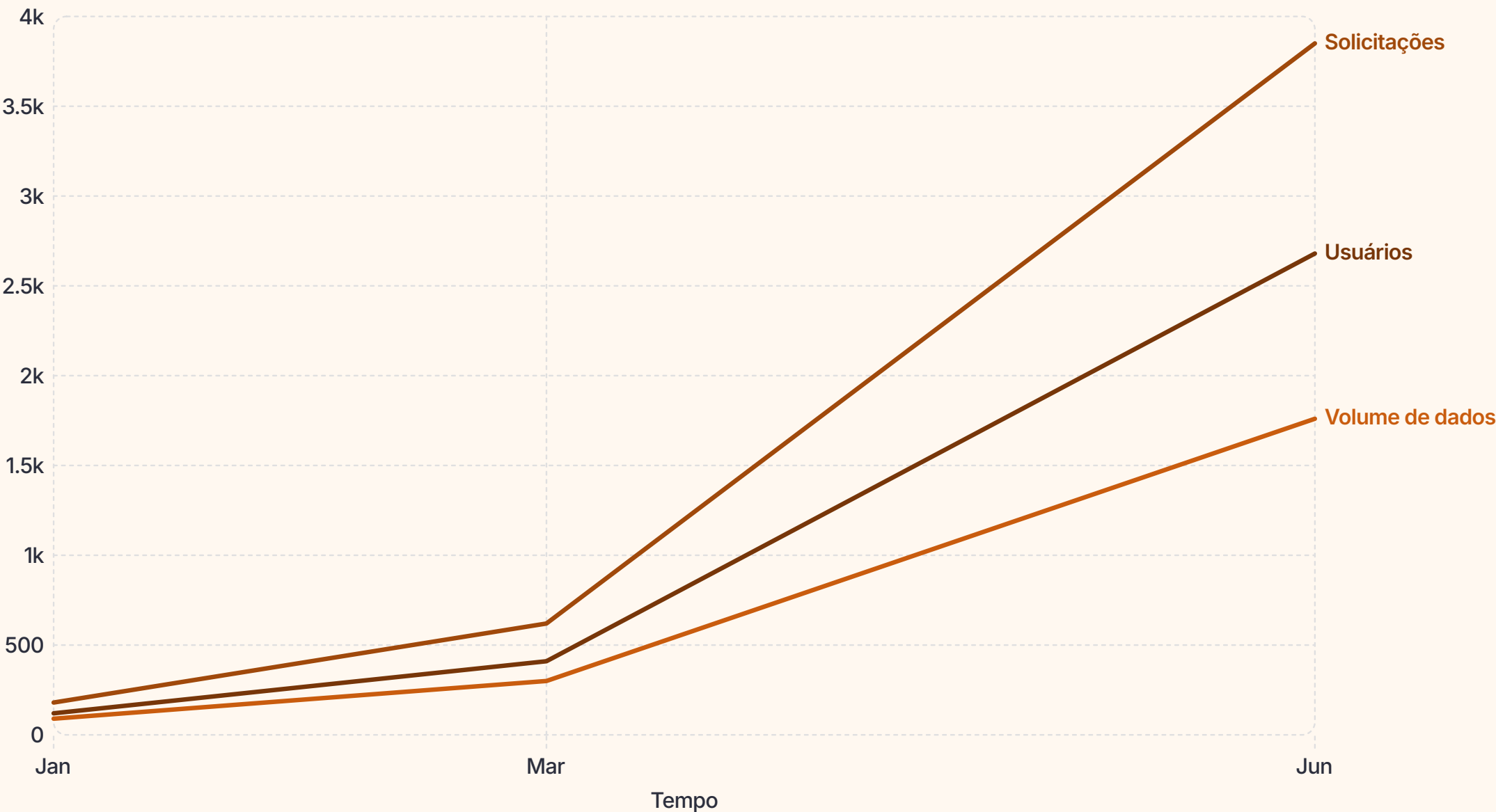
Combina simplicidade, personalização e eficiência em uma proposta única, destacando-se frente às soluções tradicionais.

Exemplo concreto

Funcionalidades pensadas para apoiar a rotina sem sobrecarregar a interface.

Escalabilidade do Produto

O TimerBook foi pensado para crescer de forma sustentável, acompanhando o aumento de usuários, reservas e operações sem perder desempenho.



À medida que a demanda aumenta, a arquitetura permite distribuir carga, ampliar capacidade e manter estabilidade. Isso garante que o produto possa evoluir com segurança, suportando mais volume sem comprometer a experiência do usuário.

Feedback dos Usuários

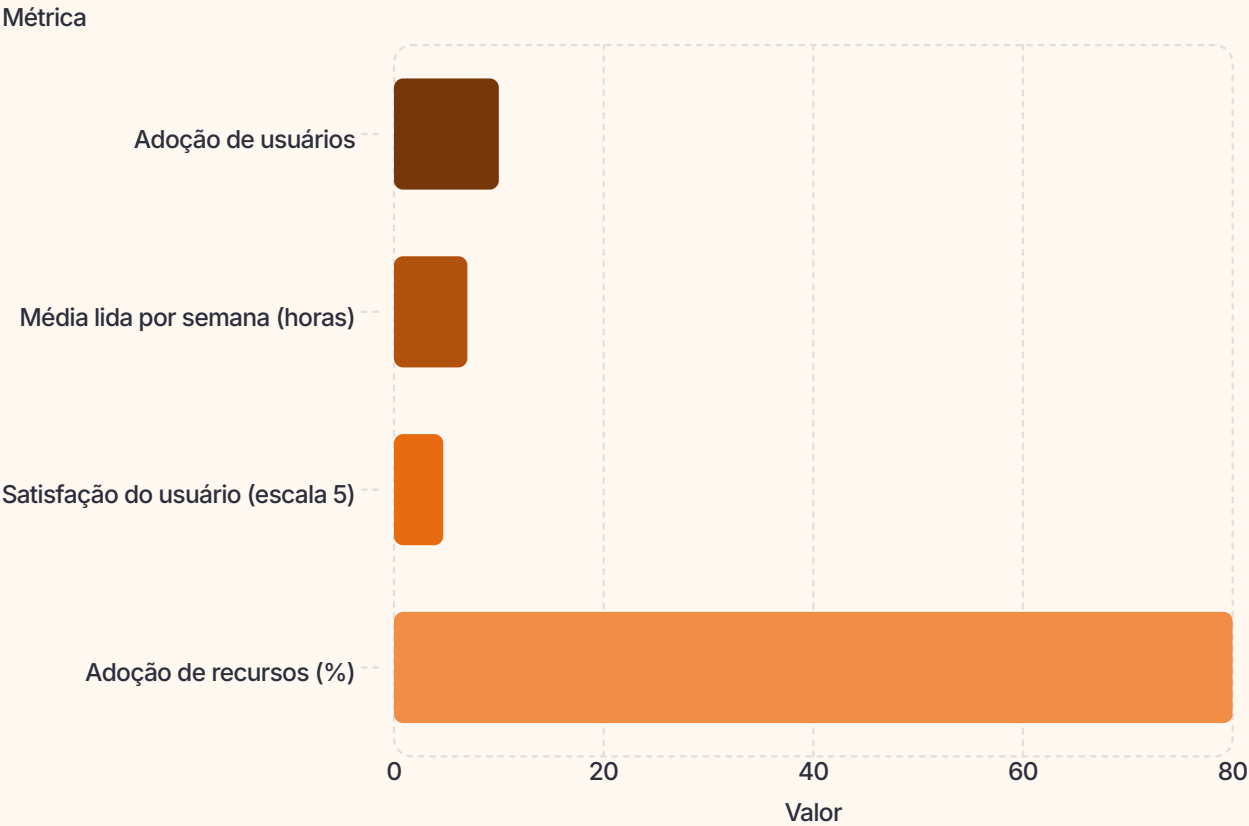
Durante os testes e a validação, o feedback foi coletado por meio de entrevistas, questionários e sessões de uso guiadas com diferentes perfis de usuários do TimerBook.

As respostas foram muito positivas: os usuários destacaram a facilidade de uso, a agilidade para fazer reservas e a clareza da experiência. Também elogiaram a organização do fluxo e a confiança transmitida pela interface.

7



Métricas de Sucesso



Indicadores avaliados

Adoção de usuários: crescimento no uso do MVP após os testes.

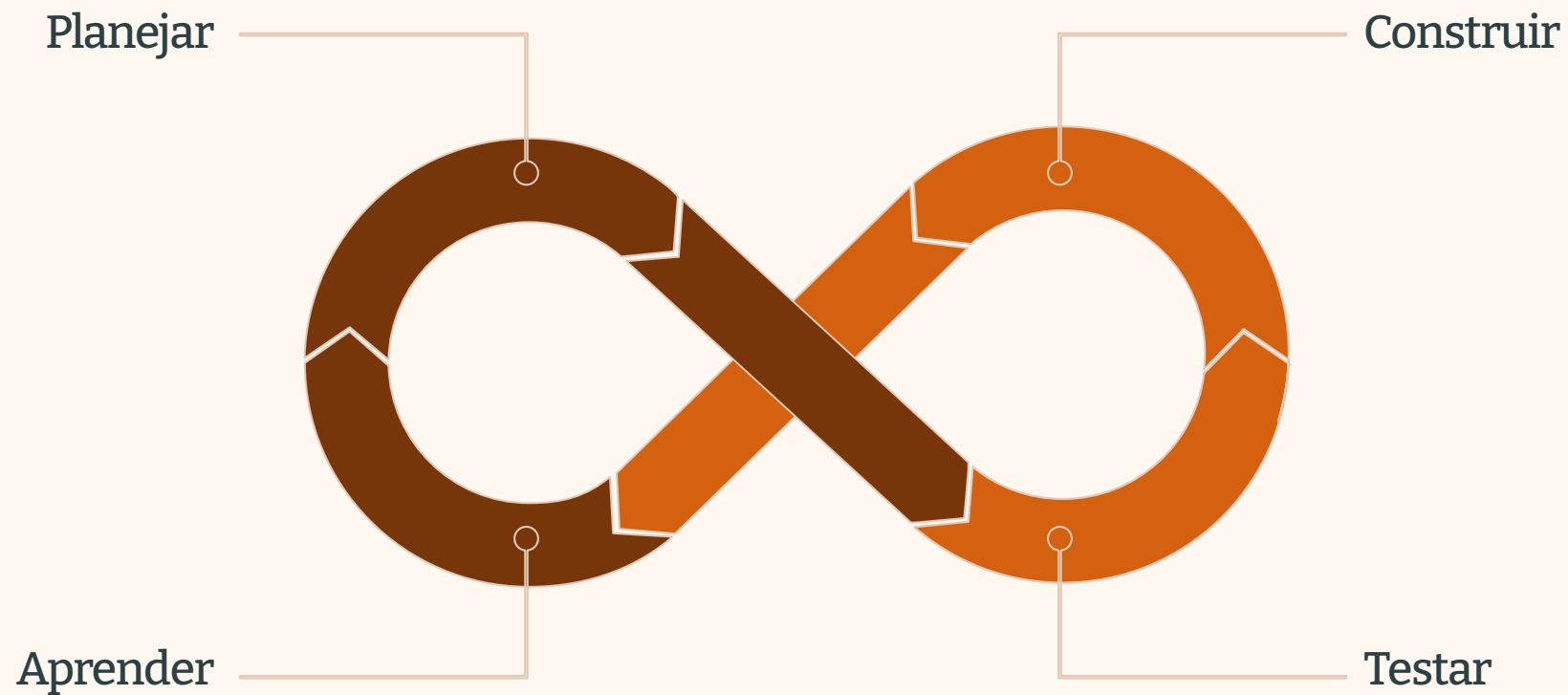
Satisfação do usuário: percepção geral da experiência.

Adoção de recursos: uso das principais funcionalidades do produto.

Média lida por semana: o quanto o usuário engajou no aplicativo

Essas métricas validam o MVP, demonstrando a aceitação do produto pelos usuários, sua estabilidade operacional e a eficácia de suas funcionalidades principais, comprovando que a solução atende às necessidades identificadas e está pronta para as próximas etapas de desenvolvimento e expansão.

Iteração e Aprendizado:



Processo de Desenvolvimento e Colaboração

Adotamos uma abordagem ágil baseada em sprints, com ciclos curtos de entrega e revisão contínua. A divisão de tarefas foi feita considerando as habilidades de cada membro e a complexidade das funcionalidades.

- 1 Planejamento**
Levantamento de requisitos, definição do backlog e priorização das histórias de usuário
- 2 Sprints**
Ciclos de 1–2 semanas com desenvolvimento, code review e integração contínua via Git
- 3 Testes e Validação**
Testes manuais e automatizados, validação com usuários reais e ajustes de usabilidade

Ferramentas

Git/GitHub, Trello, Discord

Reuniões

Dailies semanais e retrospectivas ao fim de cada sprint

Divisão

Frontend, Backend, UX/UI e Documentação distribuídos por perfil

SPRINT PLANING



Testes Realizados

No projeto TimerBook, os testes foram essenciais para garantir a qualidade, a estabilidade e a boa experiência de uso da plataforma. Eles ajudaram a validar desde regras de negócio e integrações críticas até a navegação e a resposta do sistema em cenários de uso real.

1

Testes Unitários

Validação de funções e regras isoladas do sistema, como cálculos, validações de formulários e tratamento de dados.

- Validação de campos obrigatórios
- Regras de agendamento e atualização
- Tratamento de erros em funções críticas

2

Testes de Integração

Verificação dos principais fluxos entre front-end, back-end e API para garantir que os módulos funcionam em conjunto.

- Criação e edição de registros via API
- Autenticação e acesso às rotas protegidas
- Persistência e retorno correto dos dados

Twelve-Factor App no Projeto

Aplicamos os princípios da metodologia Twelve-Factor App para garantir que o sistema seja robusto, escalável e fácil de manter. Abaixo, os fatores mais relevantes para o nosso contexto.

I — Codebase

Um único repositório Git versionado, com histórico completo de mudanças e branches organizados por funcionalidade.

II — Dependencies

Dependências declaradas explicitamente em `package.json`, garantindo instalações consistentes em qualquer ambiente.

III — Config

Credenciais, URLs e chaves de API mantidas em variáveis de ambiente, separadas do código e ajustadas por ambiente.

IV — Backing Services

Banco de dados, e-mail e APIs externas tratados como serviços anexados, acessados via configuração e sem dados hardcoded.

V — Build, release, run

O processo foi separado em build, release e execução, permitindo gerar uma versão pronta antes da publicação e do uso.

VI — Processes

A aplicação foi mantida como processos sem estado, os dados são armazenados em um banco de dados.

VII — Port binding

O sistema expõe sua interface por porta definida em configuração, permitindo execução padronizada em qualquer ambiente.

VIII — Concurrency

IX — Disposability

Inicialização e desligamento rápidos, com recuperação ágil após falhas e reinício simples em ambientes de execução.

X — Dev/prod parity

Ambientes de desenvolvimento e produção foram mantidos o mais próximos possível para reduzir diferenças e evitar surpresas no deploy.

XI — Logs

Eventos e erros enviados para saída padrão e coletados por ferramentas externas para monitoramento e diagnóstico.

XII — Admin processes

Migrations e outras tarefas administrativas, como atualização do banco com Flyway, são executadas como processos pontuais e isolados, usando a mesma configuração da aplicação, sem fazer parte da execução contínua do sistema

Tecnologias Utilizadas - Web

A escolha do stack tecnológico foi guiada por critérios de maturidade da comunidade, compatibilidade entre as ferramentas, facilidade de integração e adequação às necessidades do TimerBook, como leitura de PDFs, autenticação, estatísticas, pagamentos e recursos de inteligência artificial.



Frontend

React.js com Vite, utilizado para construir uma interface web moderna, componentizada e responsiva. O projeto também utiliza React Router DOM para navegação entre páginas, Axios para comunicação com a API, React PDF para visualização de documentos, Recharts para gráficos e Tailwind CSS/CSS para estilização.



Banco de Dados

PostgreSQL como banco de dados relacional principal, com Spring Data JPA para persistência e Flyway para controle de migrations. O banco armazena usuários, livros, leituras, sessões de leitura, conquistas, assinaturas, transações e eventos de webhook.



Backend

Java com Spring Boot, estruturado como uma API REST. O backend utiliza Spring Web, Spring Security, Spring Data JPA, Bean Validation, JWT para autenticação, OAuth2 com Google, Spring Mail para envio de e-mails e Springdoc/OpenAPI para documentação dos endpoints.

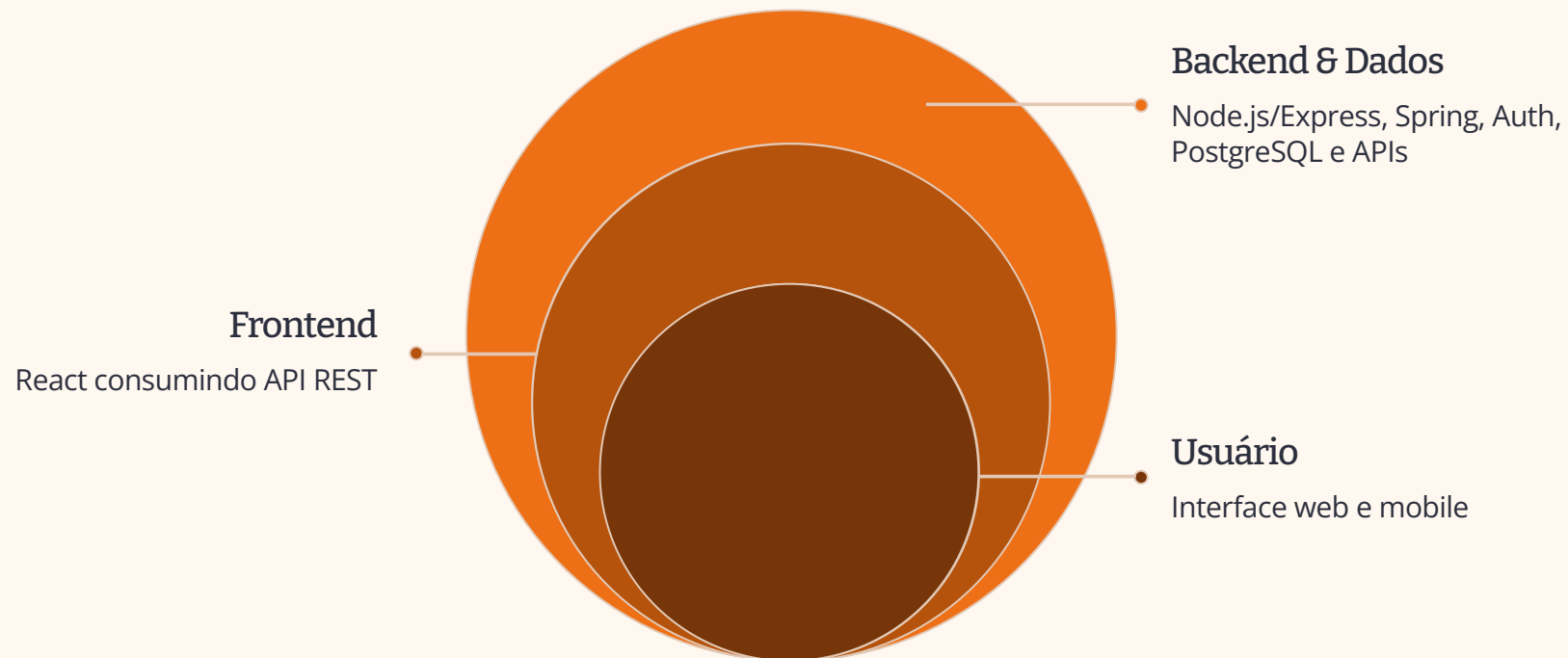


Infraestrutura

Docker e Docker Compose para organizar os serviços da aplicação, incluindo frontend, backend, PostgreSQL e pgAdmin. O projeto também possui integração com Mercado Pago para assinaturas, Ollama para recursos de IA e armazenamento de arquivos enviados em um bucket na AWS, como capas e PDFs.

Arquitetura do Software - Web

O TimerBook adota uma arquitetura cliente-servidor, com separação entre frontend, backend, banco de dados e serviços externos. Essa estrutura facilita a manutenção, a evolução do sistema e a divisão clara das responsabilidades. O diagrama acima ilustra o fluxo de dados do sistema, desde a interação do usuário até o armazenamento e integração com serviços externos.



Modelo Adotado

Arquitetura cliente-servidor com backend em camadas. API REST comunica frontend ao backend via JSON.

Principais Decisões

- Frontend e backend separados
- API REST com JWT
- Backend em camadas
- PostgreSQL com Flyway
- Armazenamento dos arquivos em um Bucket AWS
- Integração com serviços externos (Mercado Pago, Google OAuth2, Ollama)

Tecnologias Utilizadas - Mobile



Mobile

React Native com Expo para desenvolvimento multiplataforma (Android, iOS e web). Interface organizada em telas reutilizáveis com componentes próprios para botões, campos, cards de estatísticas e estados vazios.



Armazenamento em Nuvem

AWS S3 Bucket para armazenar arquivos como capas de livros, fotos de perfil e PDFs.



Integração com API

Comunicação REST centralizada usando fetch. Envia tokens JWT em requisições protegidas, realiza refresh automático e consome endpoints de login, cadastro, livros, sessões de leitura, estatísticas e perfil.



Recursos Nativos

Expo para melhorar experiência: expo-image-picker (seleção de imagens), expo-document-picker (escolha de PDFs), expo-av (reprodução de sons), WebView para leitor, SVGs para ícones e react-native-chart-kit para gráficos de estatísticas.

Arquitetura do Software - Mobile

O aplicativo mobile segue uma arquitetura cliente-servidor, consumindo a mesma API REST utilizada pelo ecossistema TimerBook. A aplicação é organizada em camadas simples: telas, componentes, serviços de API, utilitários, armazenamento local e estilos.

Modelo Adotado

- App.js como ponto central de orquestração
- Controla autenticação, tema, abas principais, carregamento de dados, sessão de leitura e navegação
- Telas separadas por responsabilidade: autenticação, início, biblioteca, novo livro, perfil, leitor e estatísticas

Fluxo Principal

- Usuário autentica no app mobile
- Tokens salvos localmente
- Aplicativo carrega dados do backend
- Usuário pode cadastrar livros com capa e PDF
- Iniciar leitura e acompanhar livros em andamento
- Visualizar estatísticas e atualizar perfil

Principais Decisões

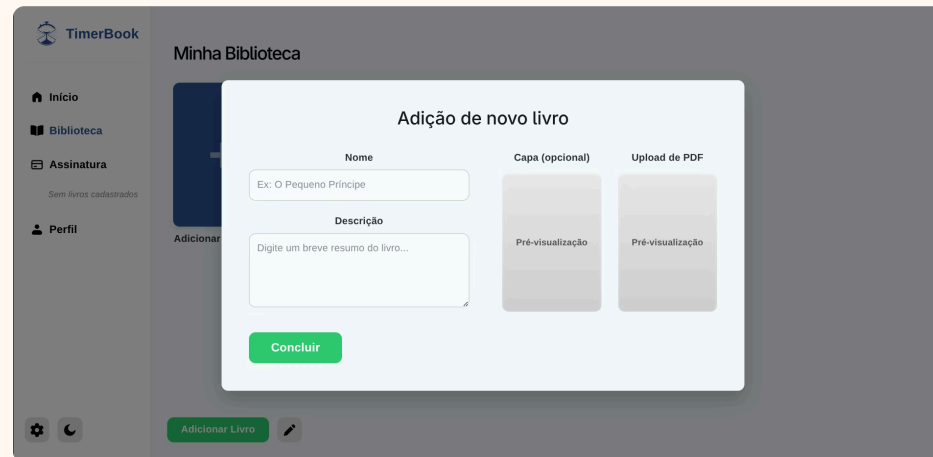
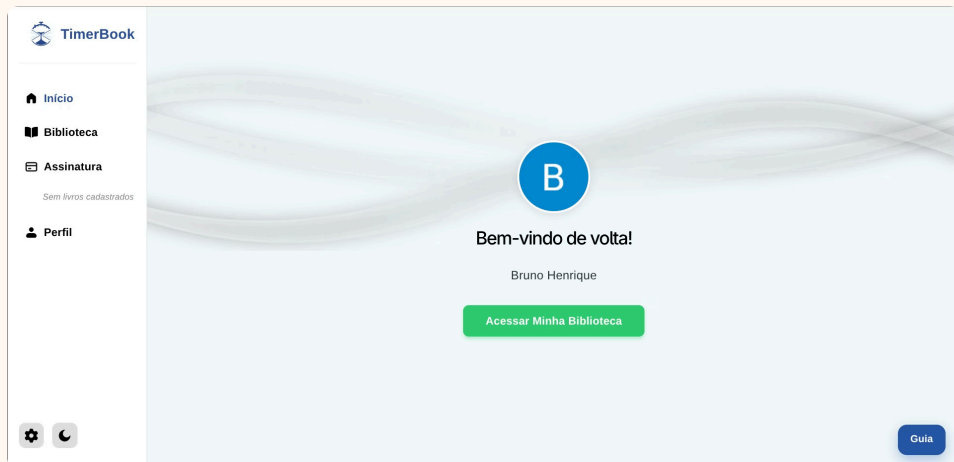
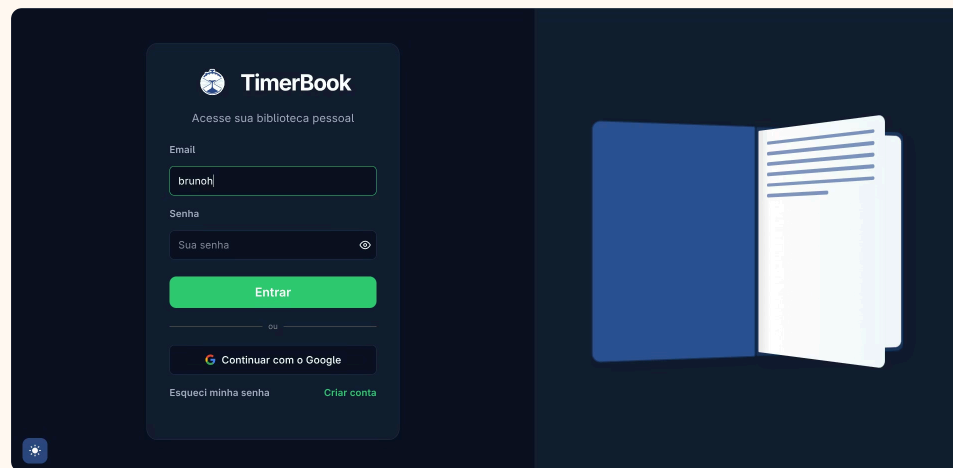
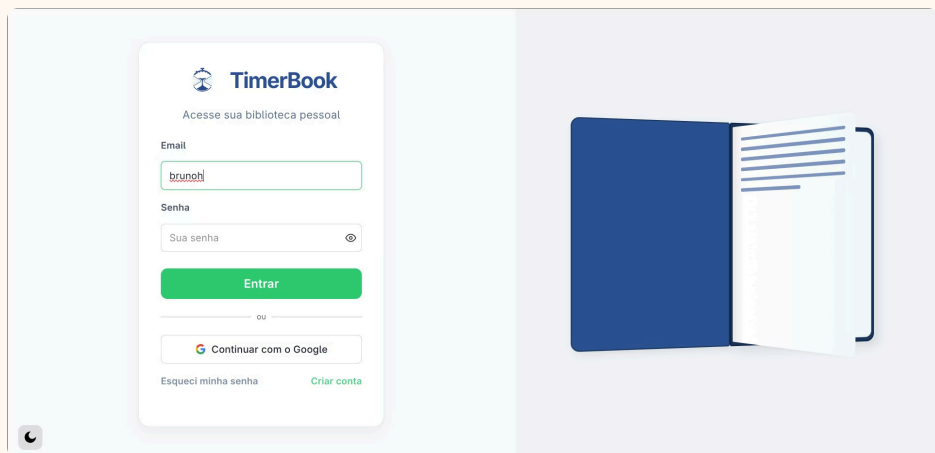
- Adoção de arquitetura cliente-servidor para integração com API REST.
- Organização da aplicação em camadas para modularidade e manutenção.
- Centralização da orquestração em App.js para controle do fluxo da aplicação.
- Separação de telas por responsabilidade para clareza e escalabilidade.
- Gestão de tokens e armazenamento local para autenticação persistente.
- Funcionalidades completas de CRUD (Create, Read, Update, Delete) para livros e sessões de leitura.

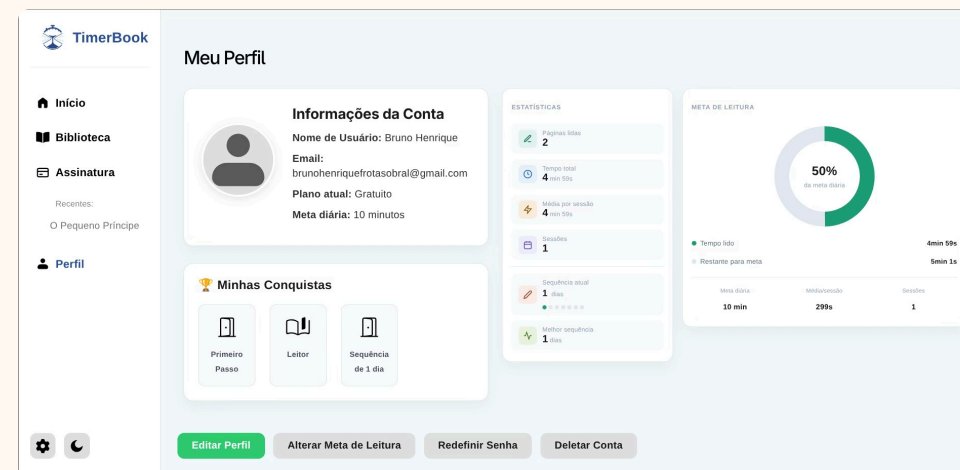
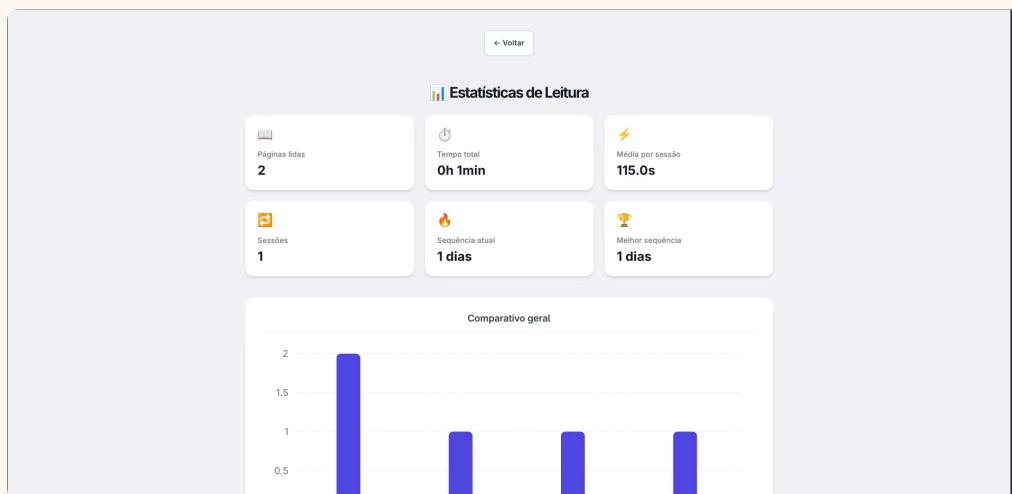
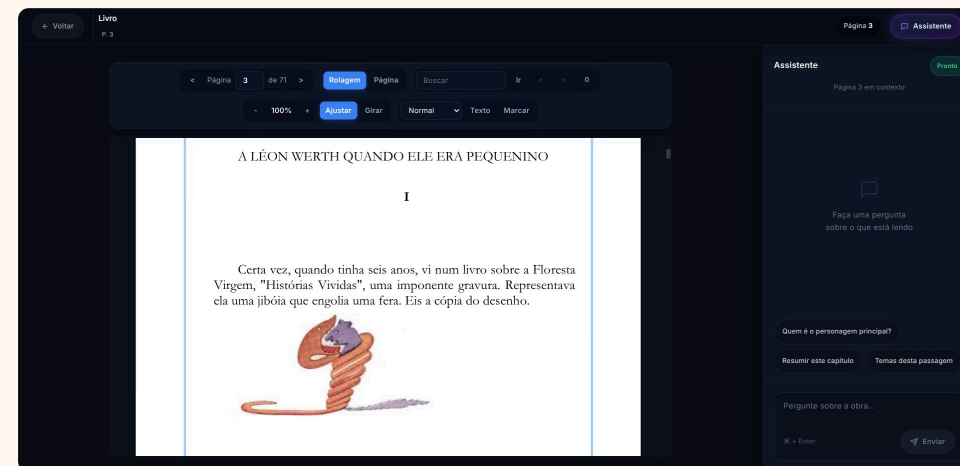
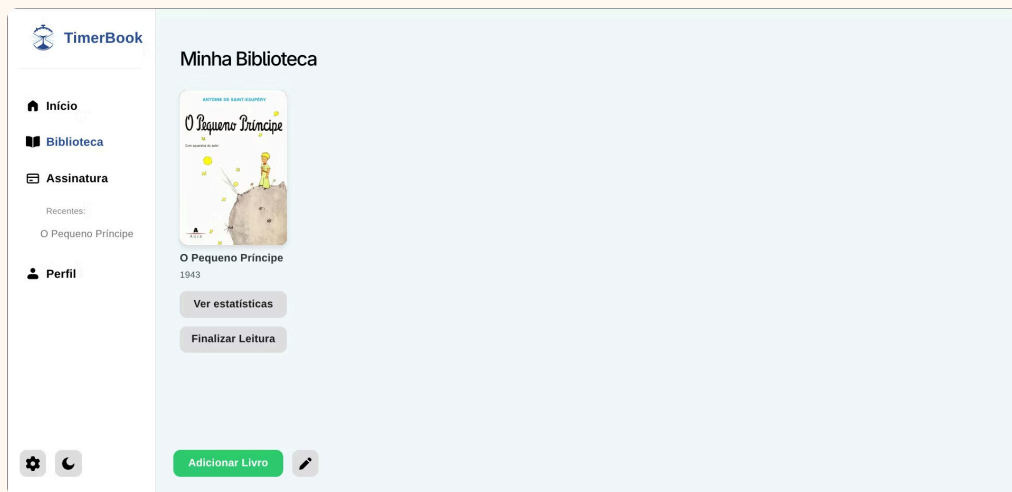
Usabilidade, UX/UI e Qualidade

Design Centrado no Usuário

Foram aplicadas várias técnicas de UX e UI para deixar o sistema agradável e fácil de usar. Os destaques são:

- Wireframes de alta fidelidade antes do desenvolvimento;
- Transição entre tema claro e escuro de forma fluida. O CSS foi ajustado para garantir que as cores tenham um bom contraste e os textos sejam sempre fáceis de ler, independente do tema escolhido;
- Consistência em todas as telas. Todos os botões, cartões de livros e janelas (modais) seguem a mesma identidade;
- Animações interativas na tela de login, menus flutuantes que aparecem na barra lateral e mensagens na tela quando o usuário atualiza o perfil com sucesso.
- Agrupamento lógico das ações na barra lateral e uso de janelas modais para manter a tela limpa e focada no acervo pessoal.







Aprendizagem baseada em projetos: um relato da turma sobre a metodologia aplicada na disciplina.

Destaquem os seguintes pontos: o processo ágil aplicado; o auto-gerenciamento das equipes; a comunicação entre os pares; e a avaliação dos pares, dentre outros pontos que considerarem relevantes.



Sugestões para a disciplina

Avaliação, Equipe e Conclusão

Avaliação do Produto

- **Proposta de valor:** Atingida — o MVP resolve o problema central identificado
- **Prontidão:** Funcional para uso piloto; melhorias previstas para v2
- **Feedback:** Usuários validaram os fluxos principais com avaliação positiva
- **Escalabilidade:** Arquitetura preparada para crescimento de usuários e dados
- **Diferencial:** [Descrever inovação ou abordagem única do projeto]

Equipe e Aprendizados

O projeto foi desenvolvido por uma equipe multidisciplinar, com divisão de responsabilidades baseada nas habilidades de cada integrante.

- **Pontos positivos:** Comunicação eficaz, entregas no prazo e colaboração ativa
- **Desafios:** Alinhamento de expectativas e gestão de dependências entre tarefas
- **Lições aprendidas:** Planejamento detalhado e documentação contínua são fundamentais
- **Sugestões:** [Incluir sugestões construtivas para a disciplina] - Web